Machine Learning — UAQ

Presentación

Angel Gutiérrez

17 de febrero de 2026

Contenido

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

Semana 1 — Introducción + workflow de Machine Learning

Framing, generalización, overfitting, leakage, baselines y métricas (Accuracy + PR/PR-AUC)

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 1

Temario de la Semana 1 (qué cubrimos hoy)

1. **Arranque y framing:** dataset $\rightarrow$ modelo $\rightarrow$ predicción $\rightarrow$ decisión
2. **Teoría (slides):**
   - Problem framing (qué es X, qué es y, unidad de predicción)
   - Generalización y overfitting
   - Leakage (conceptual + ejemplos)
   - Baselines
   - Métricas: Accuracy + Precision/Recall + PR-AUC
3. **Lab NumPy:** split + baseline + accuracy + PR básico (+ mini-leakage)
4. **Lab sklearn:** Pipeline + cross_val_score + PR-AUC
5. **Entregable:** repo + notebook plantilla + checklist reproducible

Lecturas sugeridas (PML = libro base) — Semana 1

Lectura previa (fundamentos para la sesión)

- **Cap. 1 — Introduction**
  - ¿Qué es ML? (definición tipo T/P/E) y *supervised learning*
  - *Generalization* y *overfitting* (por qué separar train/test)

Lectura mínima para métricas (lo que usaremos hoy)

- **Cap. 5 — Decision theory**
  - *Precision*, *Recall*, *Precision–Recall curve*
  - *Average Precision* (PR-AUC) como resumen

Lectura posterior (para consolidar evaluación honesta)

- **Cap. 4 — Statistics**
  - *Validation set* y selección de hiperparámetros
  - *Cross-validation* (K-fold): por qué reduce varianza con N pequeño

Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

¿Qué es "aprender" en ML? (visión de extremo a extremo)

- **Dataset** (observaciones): pares (x_i, y_i)
- **Modelo** f_θ: transforma x en una predicción $\hat{y}$
- **Métrica**: mide qué tan bien predice (Acc, AP/PR-AUC)
- **Decisión**: usar $\hat{y}$ o un score para actuar (aprobar, clasificar, alertar, etc.)

Idea central del curso

El objetivo real no es "ganar en train"; es **generalizar** con evaluación honesta y reproducible.

Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
   - Problem framing
   - Generalización y overfitting
   - Fuga de información (leakage)
   - Modelos de referencia
   - Métricas
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

Problem framing: qué debes definir antes de entrenar

- **Unidad de predicción:** ¿qué entidad scoreas? (persona, cliente, imagen, transacción)
- **Features X:** información disponible **al momento de predecir**
- **Target y:** definición operacional (evento + ventana + etiqueta)
- **Salida del modelo:** clase (0/1), probabilidad, score continuo
- **Métrica:** qué significa "éxito" (accuracy, AP, etc.)

Pregunta guía

¿Qué decisión cambia si tengo este score/predicción?

Supervised vs unsupervised vs RL (solo para ubicar)

Supervised

- Tienes etiquetas y
- Objetivo: predecir y desde x
- Ej: spam, precio de casa

Reinforcement Learning

- Aprendes por recompensa acumulada
- Interacción agente–entorno
- Ej: control, juegos

Unsupervised

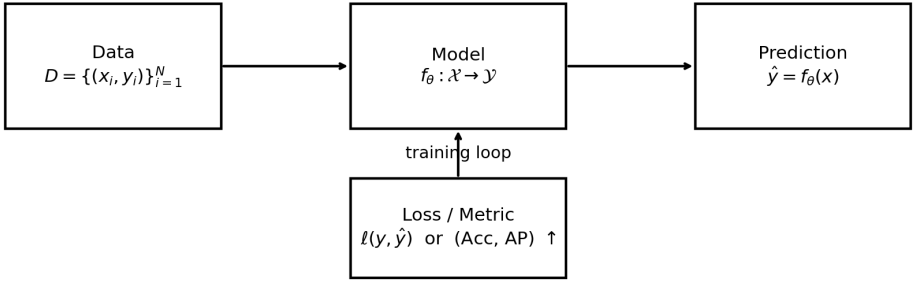
- No hay y
- Objetivo: estructura/representaciones
- Ej: clustering, PCA

Hoy

Nos enfocamos en **supervised** (clasificación binaria) + evaluación.

Supervised learning: esquema visual (para fijar ideas)

Supervised learning: learn $f(x)$ from labeled pairs (x, y)



Generalización: el objetivo real

- Queremos buen desempeño en datos nuevos (out-of-sample).
- Por eso separamos datos: train / validation / test.

Intuición

Train performance es "qué tan bien memoriza".
Test performance es "qué tan bien generaliza".

Overfitting: definición + señal práctica

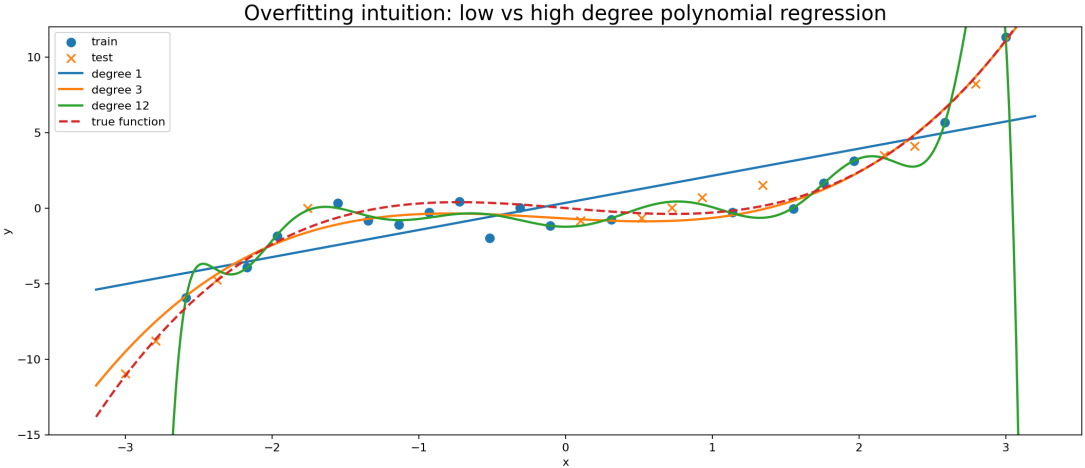
Overfitting = el modelo se ajusta demasiado a peculiaridades/ruido del train:

- train muy bien, test/val notablemente peor
- "se derrumba" al salir del dataset de entrenamiento

Generalization gap

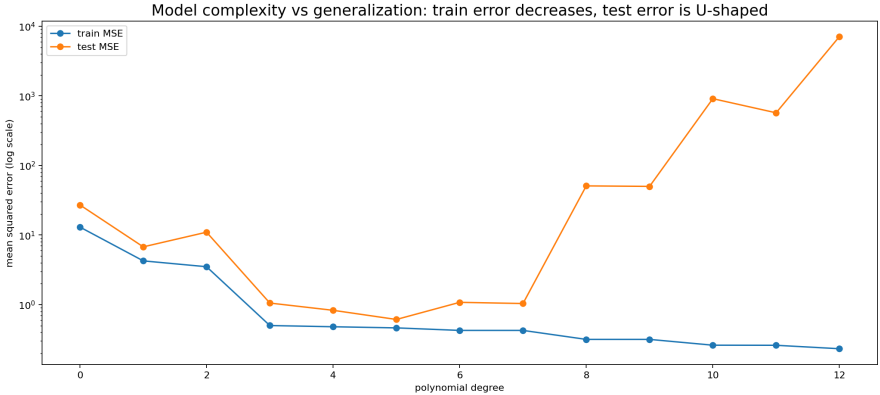
Diferencia entre el desempeño en train y en val/test.

Overfitting visual (intuición con modelos de distinta complejidad)



- Modelo simple: puede sub-ajustar (*underfit*)
- Modelo muy flexible: puede memorizar train y fallar en test

Complejidad vs error (por qué el test error es “U-shaped”)



- El **train error** suele bajar con complejidad.
- El **test error** suele tener un mínimo: demasiada complejidad vuelve a empeorar.

Fuga de información: por qué las métricas pueden mentir

Pregunta guía

¿Por qué mi métrica (accuracy / AP) es alta en el cuaderno y cae en producción?

- **Objetivo**: estimar desempeño fuera de muestra (generalización).
- **Problema**: si el proceso "ve" datos del futuro, la evaluación es optimista.
- **Consecuencia**: eliges mal el modelo / umbral $\Rightarrow$ mal despliegue.

Definición operativa (en el flujo train/val/test)

- Partición: $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}$ (disjuntos).
- En el pipeline "se aprenden":

$$\theta \text{ (modelo)}, \qquad \phi \text{ (transformaciones)}, \qquad \lambda \text{ (regularización)}, \qquad t \text{ (umbral)}.$$

Fuga de información

Ocurre si $\mathcal{D}_{\text{val/test}}$ influye (directa o indirectamente) en θ, ϕ, λ, t.

Regla de oro

Todo ajuste (`fit`) se hace solo con $\mathcal{D}_{\text{train}}$ (o dentro de cada pliegue).

Idea clave 1: cuándo el test estima el riesgo (sin sesgo)

- Riesgo (pérdida ℓ): $R(f) = \mathbb{E}[\ell(y, f(x))]$.
- Estimador en test:

$$\widehat{R}_{\text{test}}(f) = \frac{1}{n_{\text{test}}} \sum_{(x,y)\in\mathcal{D}_{\text{test}}} \ell(y, f(x)) \,.$$

Condición crítica

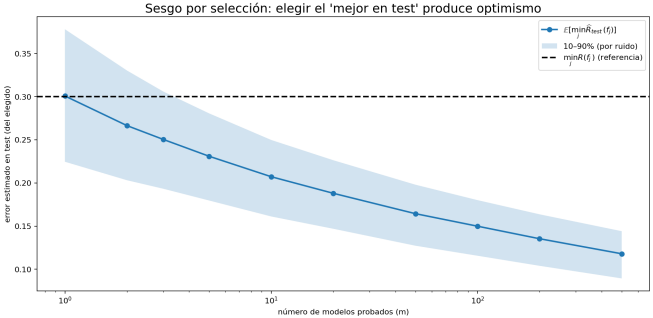
$\widehat{R}_{\text{test}}(f)$ es una evaluación honesta si f es **independiente** de $\mathcal{D}_{\text{test}}$.

- Si el test se usa para decidir f (modelo/umbral/variables), se rompe la independencia.
- $\Rightarrow$ la métrica en test se vuelve **optimista** (sesgo).

Idea clave 2: sesgo por selección (mirar test para elegir)

- Si usas $\mathcal{D}_{\text{test}}$ para elegir entre m opciones, el "ganador" suele ganar por **suerte**.
- Test finito: $\widehat{R}_{\text{test}}(f_j) = R(f_j) + \varepsilon_j$ (ruido de muestreo).
- Elegir por el mínimo introduce optimismo:

$$\widehat{R}_{\text{test}}(f_{j^\star}) = \min_j \{R(f_j) + \varepsilon_j\} \quad \text{(tiende a ser menor que } \min_j R(f_j)\text{)}.$$



Idea clave 2 (continuación): cómo evitar el sesgo por selección

Regla práctica

El test es para auditar, no para decidir.

- Decide con $\mathcal{D}_{\text{val}}$ o validación cruzada:

$$\text{modelo},\ \lambda,\ \text{variables},\ t\ (\text{umbral}).$$

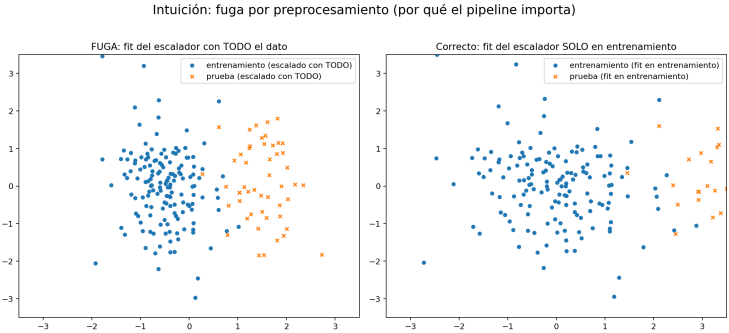
- Usa $\mathcal{D}_{\text{test}}$ **una sola vez** al final (sin "volver a mover perillas").
- Si ajustaste algo tras ver test, entonces ese test ya no es honesto $\Rightarrow$ necesitas un **nuevo test**.

Idea clave 3: fuga por preprocesamiento (por qué el pipeline importa)

- Transformación: $z = T_\phi(x)$. **Primero** se ajusta ϕ en $\mathcal{D}_{\text{train}}$.

$$\phi \leftarrow \mathrm{fit}(T, \mathcal{D}_{\text{train}}) \qquad \Rightarrow \qquad \theta \leftarrow \arg\min_\theta \widehat{R}_{\text{train}}(\theta \mid \phi).$$

- Fuga: $\phi \leftarrow \mathrm{fit}(T, \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}})$.



Ejemplo numérico mínimo: “elegí el mejor en test”

Mismo mundo, test finito

Supón que (en realidad) $R(f_A) = R(f_B) = R(f_C) = 0{,}30$.

$$\widehat{R}_{\text{test}}(f_A) = 0{,}28, \quad \widehat{R}_{\text{test}}(f_B) = 0{,}24, \quad \widehat{R}_{\text{test}}(f_C) = 0{,}27$$

- Si eliges y reportas f_B por tener 0.24, ya usaste el test para decidir.
- En un test nuevo, el rendimiento típico “regresa” cerca de 0.30.
- Solución: decide con $\mathcal{D}_{\text{val}}$ (o validación cruzada) y **audita** con $\mathcal{D}_{\text{test}}$.

Checklist: 3 patrones típicos de fuga de información

1. **Preprocesamiento**: ajustar transformaciones con todo el dataset $\Rightarrow$ ϕ debe salir solo de $\mathcal{D}_{\text{train}}$ (o por pliegue).
2. **Variable objetivo**: variables que "usan el futuro" o derivadas del desenlace $\Rightarrow$ la variable debe existir al momento de la decisión.
3. **Partición**: duplicados / mismo cliente-paciente en train y test / partición temporal incorrecta $\Rightarrow$ split por grupo o por tiempo cuando corresponda.

Señal de alarma: métrica "demasiado buena" + caída fuerte al desplegar.

Resumen: reglas prácticas (para no auto-engañarnos)

- **Test** se usa para **auditar** (una vez), no para elegir.
- **Validación / validación cruzada** se usa para decidir λ, variables y umbral t.
- Todo `fit` (modelo y preprocesamiento) vive dentro de train (o dentro del pliegue).
- La partición respeta estructura del dato: tiempo, grupos, duplicados.

Takeaway

Una métrica solo es creíble si el experimento imita el uso real del modelo.

Modelo de referencia: definición + por qué es obligatorio

Modelo de referencia = modelo simple que establece un "piso" de desempeño.

- **Referencia ingenua:** clase mayoritaria (predictor constante)
- **Referencia fuerte:** modelo lineal simple (regresión logística)

Disciplina

No celebramos un modelo si no supera una referencia razonable.

Métricas (clasificación binaria): matriz de confusión

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	TP	FN
$y = 0$	FP	TN

- TP: positivos detectados FN: positivos perdidos
- FP: falsas alarmas TN: negativos correctos

Idea

Todas las métricas de clasificación se construyen con TP, FP, FN, TN.

Precisión / Cobertura (recall): definiciones e intuición

- **Precisión** (pureza):

$$\mathrm{Prec} = \frac{TP}{TP + FP}$$

- **Cobertura** (recall):

$$\mathrm{Rec} = \frac{TP}{TP + FN}$$

Compromiso (umbral)

Al mover el umbral t, cambian $TP, FP, FN \Rightarrow$ cambia (precisión, cobertura).

Ejemplo mínimo: misma exactitud, comportamiento distinto

Dataset desbalanceado

$N = 100$: 10 positivos, 90 negativos.

Modelo A (ingenuo): $\hat{y} = 0$ siempre

$$TP = 0,\ FN = 10,\ FP = 0,\ TN = 90$$

$$\mathrm{Acc} = 0{,}90, \qquad \mathrm{Rec} = 0$$

Modelo B: detecta algunos positivos

$$TP = 6,\ FN = 4,\ FP = 6,\ TN = 84$$

$$\mathrm{Acc} = 0{,}90, \quad \mathrm{Prec} = 0{,}50, \quad \mathrm{Rec} = 0{,}60$$

Mensaje

La **exactitud** puede ocultar fallas en la clase positiva $\Rightarrow$ usamos precisión/recall y curva PR.

Exactitud (accuracy): cuándo sirve y cuándo engaña

Exactitud (accuracy):

$$\mathrm{Acc} = \frac{TP + TN}{TP + TN + FP + FN}.$$

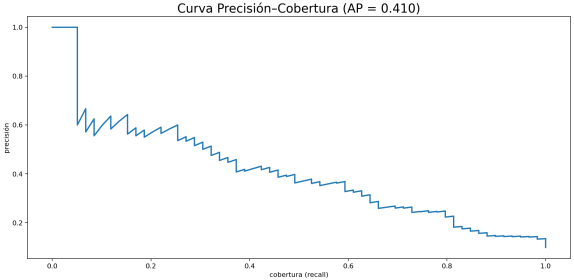
- Útil si las clases están balanceadas y los costos de error son similares.
- Puede engañar con desbalance: puedes tener alta exactitud sin detectar positivos.

Ejemplo mental

Si 5 % son positivos, predecir siempre 0 da 95 % exactitud (pero 0 recall).

Curva Precisión–Recall y AP

- Curva PR: puntos $(\mathrm{Recall}(t), \mathrm{Prec}(t))$ al variar el umbral t.
- Se usa cuando hay **desbalance** y nos importa la **clase positiva**.
- Línea base: un clasificador no-informativo tiene $\mathrm{Prec} \approx \pi$, con $\pi = P(y = 1)$.
- **AP** (Average Precision): resumen escalar tipo "área" bajo la curva PR.



Uso práctico

Elegir t según una meta: recall mínimo o precisión mínima.

Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

Lab NumPy — objetivo

Contexto: en teoría vimos **generalización**, **fuga de información**, **modelos de referencia** y **métricas** (precisión/recall, exactitud, PR/AP).

Objetivo del lab: construir ese workflow **sin librerías "mágicas"**, para entender qué se calcula y cuándo aparece la fuga.

Al final debes poder:

- generar un dataset binario sintético (X, y)
- hacer un split reproducible train/val/test
- comparar referencia ingenua vs un modelo con **score** + umbral
- medir exactitud, precisión, recall y ver el efecto del umbral
- reproducir una fuga por preprocesamiento (estandarizar "mal" vs "bien")

Archivo: `labs/week01_numpy_baseline.ipynb`

Lab NumPy — pasos (lo que deben implementar)

1. **Datos (reproducible):** genera $X \in \mathbb{R}^{n \times 2}$, $y \in \{0, 1\}$ (dos nubes / dos gaussianas) y grafica por clase.
2. **Split (sin fuga):** ‘rng.permutation‘ + cortes 60/20/20 en train/val/test (verifica tamaños y proporción de positivos).
3. **Referencia ingenua:** predice la clase mayoritaria (decidida con train) y reporta **exactitud** en val y test.
4. **Score + umbral:** define un score simple $s(x)$ y predice $\hat{y} = \mathbb{1}[s(x) \geq t]$.
5. **Barrido de umbral (clave):** evalúa varios t y arma una tabla en val con: t, exactitud, precisión, recall.
6. **Curva PR básica:** con el barrido, grafica puntos (recall, precisión) y elige t según una meta (p. ej. recall mínimo).
7. **Mini-fuga:** estandariza “con todo” vs “solo train” y compara cómo cambian métricas/curva; explica en 2–3 líneas por qué eso es fuga.

Output esperado: tabla de umbrales + gráfica PR + comentario corto (elección de t y nota sobre fuga).

Pausa

Pausa breve

Al volver: **Pipeline + cross-validation + PR-AUC**

Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

Lab sklearn — objetivo

Objetivo del lab: implementar un pipeline reproducible en `sklearn` que:

- encapsule el preprocesamiento (evitar fuga),
- evalúe de forma estable (validación cruzada),
- y reporte métricas adecuadas (accuracy y AP / PR-AUC).

Al final debes poder:

- construir `Pipeline(StandardScaler, LogisticRegression)`
- evaluar con `cross_val_score` usando:
  - `scoring=.ªccuracy"`
  - `scoring=.ªverage_precision"` (AP / PR-AUC)
- entrenar en train y auditar en test con:
  - `precision_recall_curve`
  - `average_precision_score`

Regla práctica: decisiones con validación/CV; test solo auditoría final.
Archivo: `labs/week01_sklearn_pipeline_cv.ipynb`

¿Qué es cross-validation (CV) y para qué sirve?

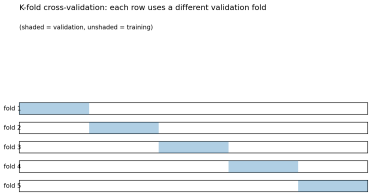
- **Definición:** dividir $\mathcal{D}_{\text{trainval}}$ en K pliegues; repetir K veces: entrenar en $K-1$ y validar en 1; promediar la métrica.
- **Pasos (en práctica):**
  - partir $\mathcal{D}_{\text{trainval}}$ en folds $F_1, \ldots, F_K$
  - para $k = 1, \ldots, K$: `fit` en $\mathcal{D}_{\text{trainval}} \setminus F_k$, evaluar en F_k
  - promediar (y reportar variabilidad) de la métrica
  - si hay hiperparámetros (λ, C): elegir el que maximiza el promedio en CV
- **Sirve para:** estimar desempeño con menor varianza y **seleccionar** modelo/hiperparámetros (p. ej. λ, C).
- **No reemplaza al test:** el test se guarda para **auditar una sola vez** al final.
- **Cuidado con fuga:** el `Pipeline` asegura que cada pliegue haga `fit` del preprocesamiento solo con su train.

Regla práctica

Decisiones con CV; test solo auditoría final.

¿Por qué cross-validation ayuda? (intuición)

- Con pocos datos, un solo split (train/val) puede dar una métrica **muy variable**.
- K-fold CV promedia el rendimiento sobre K particiones $\Rightarrow$ estimación más **estable**.
- Para selección de modelo/hiperparámetros: elegir el que maximiza la métrica promedio en CV.
- El **test** queda fuera: sirve para auditar el sistema final (sin volver a ajustar).



Discusión guiada (al final del lab)

Preguntas

- ¿Cuándo usarías accuracy y cuándo AP/PR-AUC?
- Si hay desbalance, ¿qué te dice AP que no te dice accuracy?
- ¿Qué evita Pipeline que es fácil "romper" si haces preprocessing a mano?

Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

Checklist final (lo que debe quedar bien hoy)

1. Split correcto (sin contaminar test)
2. Baseline reportado (y comparado)
3. Métrica correcta (accuracy + PR/AP)
4. Evitar leakage (Pipeline / fit solo en train)
5. Reproducibilidad (semilla + requirements + README)

Entregable: repo + notebook plantilla

- Estructura mínima:
  - `slides/week01_intro_workflow.tex`
  - `slides/figs/*.png` (figuras generadas)
  - `labs/week01_numpy_baseline.ipynb`
  - `labs/week01_sklearn_pipeline_cv.ipynb`
  - `README.md` + `requirements.txt` (o `environment.yml`)
- Evidencia mínima:
  - tabla baseline vs logistic (Accuracy + AP)
  - PR curve (imagen) + interpretación (5–6 líneas)

Cierre: 3 ideas para llevarte

1. **Generalización** es el objetivo; train perfecto no impresiona.
2. **Leakage** es el enemigo silencioso; **Pipeline** es tu escudo.
3. **Baselines + métricas correctas** te mantienen honesto.

Semana 2 — Probabilidad (modelos univariados) + Bayes

Bayes, Bernoulli/Binomial, Beta, conjugación y Beta–Binomial (posterior + predictiva)

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 2

Lecturas sugeridas (para estos temas)

- **Capítulo 2 — Probability: Univariate Models**
  - **2.3** Bayes' rule
  - **2.4** Bernoulli and binomial distributions
  - **2.7.4** Beta distribution
- **Capítulo 4 — Statistics**
  - **4.6.1** Conjugate priors
  - **4.6.2** The beta-binomial model
  - **4.6.2.4** Posterior
  - **4.6.2.9** Posterior predictive
  - **4.6.2.10** Marginal likelihood

Temario de la Semana 2 (qué cubrimos hoy)

- **Teoría (PML)**
  - **Teorema de Bayes** y **piezas** (*prior*, *likelihood*, *posterior*, *evidencia*) **(2.3)**
  - **Modelos Bernoulli y Binomial** **(2.4)**
  - **Distribución Beta** (prior sobre probabilidades) **(2.7.4)**
  - **Conjugación** (por qué Beta es conjugada) **(4.6.1)**
  - **Modelo Beta–Binomial**: posterior y posterior predictiva **(4.6.2)**
- **Lab NumPy**
  - **Beta–Bernoulli**: posterior (actualización por conteos)
  - **Posterior predictiva**: 1 ensayo y M ensayos (simulación)
- **Lab sklearn**
  - **Umbral** (τ) y reglas de decisión
  - **Métricas**: Accuracy vs Precision/Recall vs Average Precision
- **Entregable**

¿Por qué es relevante el enfoque Bayesiano?

- **Objetivo:** inferir $\theta = \Pr(y = 1)$ (una **probabilidad**) a partir de datos D.
- **Incertidumbre explícita:** obtenemos una distribución $p(\theta \mid D)$, no solo un punto.
- **Actualización coherente:** prior + datos $\Rightarrow$ posterior.
- **Predicción robusta:** la **posterior predictiva** integra incertidumbre $\Rightarrow$ menos sobreconfianza con pocos datos.
- **Regularización interpretable:** priors como **pseudoconteos** (suavizado).
- **Comparación de modelos/priors:** evidencia $p(D) =$ **probabilidad marginal de los datos**.

Enfoque frecuentista (ejemplo: tasa de compra)

- Datos: $n = 20$, $k = 3 \Rightarrow \hat{\theta} = 0{,}15$.
- Estimación: $\hat{\theta} = k/n$ (MLE).
- IC 95 % (aprox. normal):

$$\hat{\theta} \pm 1{,}96\sqrt{\frac{\hat{\theta}(1-\hat{\theta})}{n}} = 0{,}15 \pm 0{,}156 \Rightarrow [-0{,}006,\ 0{,}306]$$

- Intervalo válido en $[0, 1]$: $[0,\ 0{,}306]$.
- Interpretación: 95 % de los ICs (en repeticiones) contienen a θ.

Enfoque Bayesiano (mismo ejemplo)

- Prior: $\theta \sim \mathrm{Beta}(2, 8)$ (creencia previa $\approx 0{,}20$).
- Posterior: $\theta \mid D \sim \mathrm{Beta}(5, 25)$.
- Media posterior: $\mathbb{E}[\theta \mid D] = \dfrac{5}{30} \approx 0{,}167$.
- Intervalo creíble 95 %: percentiles de $\mathrm{Beta}(5, 25)$.
- Interpretación: $P(\theta \in \mathrm{ICr} \mid D) = 0{,}95$.

Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

Motivación: inferir una probabilidad con pocos datos

- Problema base: estimar $\theta = \Pr(y = 1)$ a partir de pocos ensayos Bernoulli.
- Bayes da **distribución** sobre θ, no solo un número.
- Ventaja: incorpora información previa y cuantifica incertidumbre.
- Caso canónico: Bernoulli/Binomial + Beta $\Rightarrow$ actualizaciones cerradas.

Definiciones (2.3): piezas de Bayes

- **Prior:** $p(\theta)$.
- **Likelihood:** $p(D \mid \theta)$.
- **Posterior:** $p(\theta \mid D) = \dfrac{p(D \mid \theta)\, p(\theta)}{p(D)}$.
- **Evidencia:** $p(D) = \int p(D \mid \theta)\, p(\theta)\, d\theta$ (*marginal likelihood*).

Derivación 1 (2.4): likelihood Bernoulli/Binomial

- Datos iid $y_i \in \{0, 1\}$, con $k = \sum_{i=1}^{n} y_i$.
- Bernoulli: $p(y_i \mid \theta) = \theta^{y_i}(1-\theta)^{1-y_i}$.
- Likelihood agregada:

$$p(D \mid \theta) = \prod_{i=1}^{n} \theta^{y_i}(1-\theta)^{1-y_i} = \theta^k(1-\theta)^{n-k}.$$

- Binomial: $p(k \mid n, \theta) = \binom{n}{k}\theta^k(1-\theta)^{n-k}$.

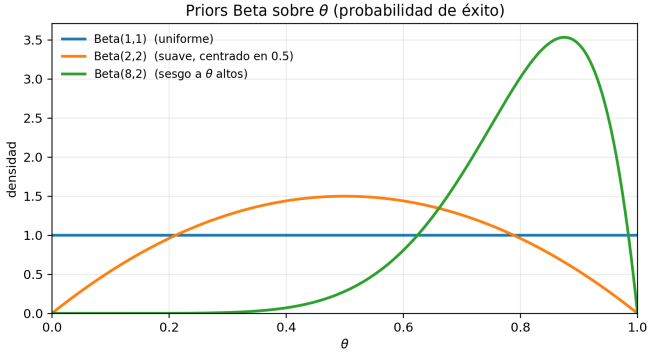
Distribución Beta (2.7.4): prior sobre probabilidades

- Soporte: $\theta \in (0,1)$ (ideal para probabilidades).
- Densidad:

$$\theta \sim \mathrm{Beta}(\alpha, \beta), \qquad p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}.$$

- Lectura: α, β controlan **sesgo** (media) y **fuerza** (concentración).
- Interpretación útil: α, β se comportan como **pseudoconteos** de éxitos/fracasos.

Priors Beta: ¿qué "forma" de creencia estás poniendo?



- $\mathrm{Beta}(1,1)$: uniforme.
- $\mathrm{Beta}(2,2)$: suave, centrada en 0,5.
- $\mathrm{Beta}(8,2)$: empuja hacia θ altos.
- **Idea:** el prior dice qué valores de θ son "plausibles" antes de ver datos.

Derivación 2 (4.6.1): conjugación Beta con Bernoulli/Binomial

- Usamos: $p(\theta \mid D) \propto p(D \mid \theta)\, p(\theta)$.
- Como $p(D \mid \theta) \propto \theta^k (1-\theta)^{n-k}$ y $p(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$, el posterior queda:

$$p(\theta \mid D) \propto \theta^{k+\alpha-1}(1-\theta)^{n-k+\beta-1}.$$

- Conjugación (misma familia):

$$\theta \mid D \sim \mathrm{Beta}(\alpha + k,\ \beta + n - k).$$

- Lectura rápida: k suma a "éxitos" y $n-k$ suma a "fracasos".

¿Qué es la **posterior predictiva**?

- **Definición:** distribución de **datos futuros** dado lo observado:

$$p(D_{\text{new}} \mid D) = \int p(D_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

- **Idea:** en vez de fijar θ a un punto (plug-in), **promediamos** sobre la incertidumbre en θ.
- En Bernoulli/Binomial:
  - 1 paso: $p(y_{\text{new}} = 1 \mid D) = \mathbb{E}[\theta \mid D]$.
  - M pasos: $S = \sum_{j=1}^{M} y_j \mid D$ tiene una distribución (Beta–Binomial) más "ancha" que plug-in.

Predictiva *plug-in* vs predictiva Bayesiana (en una línea)

- **Plug-in:** primero estimas $\hat{\theta}$ (p.ej. MLE/MAP/media posterior) y luego predices como si fuera cierto:

$$p_{\text{plug-in}}(D_{\text{new}} \mid D) = p(D_{\text{new}} \mid \hat{\theta}).$$

- **Bayes:** no fijas θ; promedias con el posterior:

$$p(D_{\text{new}} \mid D) = \int p(D_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

- **Mensaje:** plug-in suele ser más "optimista"; Bayes tiende a ser menos sobreconfiante con pocos datos.

Modelo Beta–Binomial (4.6.2): posterior y predictiva

- Predictiva 1 paso:

$$p(y_{\text{nuevo}} = 1 \mid D) = \mathbb{E}[\theta \mid D] = \frac{\alpha + k}{\alpha + \beta + n}.$$

- Predictiva de conteos (m futuros):

$$p(k_{\text{new}} \mid m, D) = \binom{m}{k_{\text{new}}} \frac{B(\alpha + k + k_{\text{new}}, \beta + n - k + m - k_{\text{new}})}{B(\alpha + k, \beta + n - k)}.$$

Ejemplo numérico mínimo

- Prior uniforme: $\theta \sim \mathrm{Beta}(1, 1)$.
- Datos: $n = 4$ lanzamientos, $k = 3$ éxitos.
- Posterior: $\theta \mid D \sim \mathrm{Beta}(4, 2)$.
- Predictiva: $p(y_{\text{nuevo}} = 1 \mid D) = \dfrac{4}{6} = \dfrac{2}{3}$.

Derivación (4.6.2.10): evidencia Beta–Binomial $p(k \mid n, \alpha, \beta)$

- Definición (marginalizar θ):

$$p(k \mid n, \alpha, \beta) = \int_0^1 p(k \mid n, \theta)\, p(\theta \mid \alpha, \beta)\, d\theta.$$

- Sustituimos Binomial y Beta:

$$p(k \mid n, \theta) = \binom{n}{k} \theta^k (1-\theta)^{n-k}, \quad p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}.$$

- Agrupando potencias:

$$p(k \mid n, \alpha, \beta) = \binom{n}{k} \frac{1}{B(\alpha, \beta)} \int_0^1 \theta^{\alpha+k-1} (1-\theta)^{\beta+n-k-1}\, d\theta.$$

- Reconocemos la integral como $B(\alpha + k, \beta + n - k) \Rightarrow$

$$p(k \mid n, \alpha, \beta) = \binom{n}{k} \frac{B(\alpha + k, \beta + n - k)}{B(\alpha, \beta)}.$$

Evidencia $p(D)$: intuición y comparación de modelos

- **Qué es:** $p(D)$ es la **probabilidad marginal** del dataset (también *marginal likelihood*).
- **Rol en Bayes:** es el **normalizador** (no depende de θ).
- **Intuición:** promedia el likelihood sobre θ "plausibles" según el prior.
- **Comparación:** para dos modelos/priors M_1, M_2,

$$\text{Bayes factor} = \frac{p(D \mid M_1)}{p(D \mid M_2)}.$$

- **Lectura:** favorece modelos que explican bien los datos **sin** "apostar" probabilidad en demasiados θ.

Evidencia: misma data, distinto prior $\Rightarrow$ distinto $p(D)$

- Datos (Binomial): $n = 10$, $k = 7$ éxitos.
- Con prior $\theta \sim \mathrm{Beta}(\alpha, \beta)$:

$$p(k \mid n, \alpha, \beta) = \binom{n}{k} \frac{B(\alpha + k, \beta + n - k)}{B(\alpha, \beta)}.$$

- Evidencias (aprox.):
  - $\mathrm{Beta}(1, 1)$: $p(k \mid n) \approx 0{,}0909$
  - $\mathrm{Beta}(2, 2)$: $p(k \mid n) \approx 0{,}1119$
  - $\mathrm{Beta}(8, 2)$: $p(k \mid n) \approx 0{,}1486$
- **Interpretación (en simple):** estos números dicen qué tan "esperable" era ver $7/10$ *antes* de ver los datos, según cada prior.
- **Lo que importa: comparaciones.** Por ejemplo:

$$\frac{p(D \mid \mathrm{Beta}(8, 2))}{p(D \mid \mathrm{Beta}(1, 1))} \approx 1{,}63, \qquad \frac{p(D \mid \mathrm{Beta}(2, 2))}{p(D \mid \mathrm{Beta}(1, 1))} \approx 1{,}23.$$

Resumen y takeaways

- Bayes (2.3) combina prior y likelihood para obtener $p(\theta \mid D)$.
- Bernoulli/Binomial (2.4) $\Rightarrow$ likelihood $\theta^k(1-\theta)^{n-k}$.
- Beta (2.7.4) $\Rightarrow$ prior natural sobre probabilidades.
- Conjugación (4.6.1): $\mathrm{Beta}(\alpha, \beta) \to \mathrm{Beta}(\alpha + k, \beta + n - k)$.
- Predictiva (4.6.2): promedia sobre $\theta \Rightarrow$ menos sobreconfianza.
- Evidencia (4.6.2.10): $p(D)$ sirve para **comparar** priors/modelos (Bayes factor).

Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

Lab NumPy — objetivos (qué debes lograr)

- **Inferencia:** a partir de $y \in \{0,1\}^n$, calcular N_1, N_0 y actualizar $(\alpha, \beta) \to (\alpha', \beta')$.
- **Predicción:** calcular $p(y_{\text{new}} = 1 \mid D)$ y aproximar $p(S \mid D)$ para M futuros.
- **Comparación:** contrastar predictiva *plug-in* vs *Bayesiana* y explicar el efecto del prior (sesgo y fuerza) en 2–3 frases.

Lab NumPy — definiciones mínimas (puente con Teoría)

- **Parámetro:** $\theta = \Pr(y = 1)$.
- **Datos:** $D = \{y_i\}_{i=1}^{n}$, con conteos $N_1 = \sum_i y_i$, $N_0 = n - N_1$.
- **Prior:** $\theta \sim \mathrm{Beta}(\alpha, \beta)$.
- **Posterior (conjugación):** $\theta \mid D \sim \mathrm{Beta}(\alpha', \beta')$, donde $\alpha' = \alpha + N_1$, $\beta' = \beta + N_0$.
- **Posterior predictiva:** $p(D_{\text{new}} \mid D) = \int p(D_{\text{new}} \mid \theta) p(\theta \mid D) d\theta$.
- **Plug-in vs Bayes:** plug-in fija $\theta = \hat{\theta}$; Bayes integra sobre θ.

Lab NumPy — Pregunta 1: ¿Qué datos tengo y qué resumen necesito?

```
ENTRADA: y[1..n]  (valores 0/1)

N1 <- suma(y)
N0 <- n - N1

SALIDA: N1, N0
CHEQUEO: N1 + N0 = n
```

- Definición: N_1 = número de éxitos, N_0 = número de fracasos.

Lab NumPy — Pregunta 2: ¿Qué creía antes de ver datos? (prior)

```
ELIGE el prior (alpha > 0, beta > 0)

EJEMPLOS:
  alpha = 1, beta = 1   (uniforme)
  alpha = 2, beta = 2   (suave, centrado en 0.5)
  alpha = 8, beta = 2   (sesgo hacia theta altos)
```

- Tip: $\alpha + \beta$ controla la "fuerza" del prior.
- Pitfall: prior fuerte sin justificar.

Lab NumPy — Pregunta 3: ¿Qué creo después de ver datos? (posterior)

```
ACTUALIZA (conjugacion Beta--Bernoulli):
alpha_post <- alpha + N1
beta_post  <- beta  + N0

SALIDA: alpha_post, beta_post
CHEQUEO: alpha_post > 0 y beta_post > 0
```

- Definición: $\alpha' = \alpha + N_1$, $\beta' = \beta + N_0$.

Lab NumPy — Pregunta 4: ¿Qué predigo del siguiente ensayo? (predictiva 1 paso)

```
PREDICTIVA POSTERIOR (1 ensayo):
p1_bayes <- alpha_post / (alpha_post + beta_post)
p0_bayes <- 1 - p1_bayes

SALIDA: p(y_nuevo = 1 | D) = p1_bayes
```

- Definición: $p(y_{\text{new}} = 1 \mid D) = \mathbb{E}[\theta \mid D] = \dfrac{\alpha'}{\alpha' + \beta'}$.

Lab NumPy — Pregunta 5: ¿Qué predigo de M ensayos futuros? (predictiva de conteos)

```
FIJA M (ensayos futuros) y R (numero de simulaciones)

PARA r = 1..R:
  muestrea theta_r ~ Beta(alpha_post, beta_post)
  muestrea S_r ~ Binomial(M, theta_r)

ESTIMA p(S | D) con {S_r} (histograma / frecuencias)
CHEQUEO: suma_s p_hat(S=s | D) ~= 1
```

- Definición: $S = \sum_{j=1}^{M} y_j =$ número de éxitos futuros en M ensayos.
- Lógica Bayesiana: muestrear $\theta \sim \mathrm{Beta}(\alpha', \beta')$ y luego $S \sim \mathrm{Binomial}(M, \theta)$.

Lab NumPy — Pregunta 6: Predictiva plug-in vs Bayesiana

Objetivo: Comparar dos formas de predecir S = número de éxitos en M ensayos futuros.

	S_bayes	**S_plugin**
Qué es	Muestras de $p(S \mid D)$	Muestras de $\mathrm{Bin}(M, \hat{\theta})$
Cómo se obtiene	$\theta \sim \mathrm{Beta}(\alpha', \beta')$, luego $S \sim \mathrm{Bin}(M, \theta)$	$S \sim \mathrm{Bin}(M, \theta_{\mathrm{MAP}})$ con θ fijo
Incertidumbre sobre θ	Sí (integra sobre la posterior)	No (usa un solo $\hat{\theta}$)

Pasos:

1. **Estimador puntual:** θ_{MAP}. Si $\alpha' > 1$ y $\beta' > 1$: $\dfrac{\alpha' - 1}{\alpha' + \beta' - 2}$; si no: $\dfrac{\alpha'}{\alpha' + \beta'}$.
2. **S_plugin:** para $r = 1, \ldots, R$, muestrea $S_r \sim \mathrm{Binomial}(M, \theta_{\mathrm{MAP}})$.
3. **S_bayes:** para $r = 1, \ldots, R$, muestrea $\theta_r \sim \mathrm{Beta}(\alpha', \beta')$, luego $S_r \sim \mathrm{Binomial}(M, \theta_r)$.
4. **Comparar:** media, varianza y una probabilidad de cola (ej. $P(S \leq k)$ o $P(S \geq k)$) de S_plugin vs S_bayes. Opcional: histogramas superpuestos.

Interpretación: S_bayes suele tener más varianza y colas más pesadas; el plug-in suele subestimar la

Lab NumPy — entregables (checklist)

Tu notebook debe incluir (evaluable):

- **Resultados:** (α, β), (α', β'), N_1, N_0, y $p(y_{\text{new}} = 1 \mid D)$.
- **Predictiva:** aproximación de $p(S \mid D)$ para algún M (elige M y R y repórtalos).
- **Comparación:** Bayes vs plug-in (una figura o, mínimo, media/varianza y una probabilidad de cola).
- **Discusión (2–3 frases):** qué cambia al variar $\alpha + \beta$ (fuerza) y la media del prior (sesgo).
- **Reproducible:** semilla fija y celdas sin salidas gigantes.

Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

Lab sklearn — objetivo

En un problema binario:

- entrenar un modelo que produzca $\hat{p}$
- barrer umbrales τ y ver trade-offs
- comparar accuracy vs precision/recall vs average precision

Lab sklearn — Paso 1: modelo con probabilidades

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# X, y: tus datos (numpy arrays)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

clf = LogisticRegression(max_iter=1000)
clf.fit(Xtr, ytr)

p_hat = clf.predict_proba(Xte)[:, 1]  # scores en [0,1]
```

Lab sklearn — Paso 2: umbral y conteos TP/FP/TN/FN

```
def yhat_from_tau(p, tau):
    return (p >= tau).astype(int)

tau = 0.5
yhat = yhat_from_tau(p_hat, tau)

TP = int(((yhat==1) & (yte==1)).sum())
FP = int(((yhat==1) & (yte==0)).sum())
TN = int(((yhat==0) & (yte==0)).sum())
FN = int(((yhat==0) & (yte==1)).sum())
print(TP, FP, TN, FN)
```

Lab sklearn — Paso 3: métricas por umbral

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

taus = np.linspace(0.0, 1.0, 101)
rows = []
for tau in taus:
    yhat = (p_hat >= tau).astype(int)
    acc = accuracy_score(yte, yhat)
    prec = precision_score(yte, yhat, zero_division=0)
    rec  = recall_score(yte, yhat, zero_division=0)
    rows.append([tau, acc, prec, rec])

rows = np.array(rows)
best_tau_acc = rows[rows[:,1].argmax(), 0]
print("best tau (accuracy):", best_tau_acc)
```

- Pitfall: escoger τ usando el test (se elige con validación).

Lab sklearn — Paso 4: Average Precision (AP) y curva PR

```
from sklearn.metrics import average_precision_score, precision_recall_curve

ap = average_precision_score(yte, p_hat)
prec, rec, thr = precision_recall_curve(yte, p_hat)

print("Average Precision =", ap)
# (Opcional) graficar: rec (x) vs prec (y)
```

Lab sklearn — preguntas guía

- ¿Qué pasa con precision y recall cuando subes τ?
- ¿Por qué accuracy puede ser engañosa con desbalance?
- ¿Qué te dice AP que no te dice un umbral fijo?

Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

Entregable (Semana 2)

Subir al repo:

- **Notebook 1 (NumPy):** Beta–Bernoulli posterior + posterior predictiva (1 y M).
- **Notebook 2 (sklearn):** umbrales + tabla de métricas + AP (y opcional curva PR).
- **README corto:** cómo correr, qué aprendiste (5–8 líneas) y una figura/tabla destacada.

Checklist de entrega (formato)

- Carpeta: `week02/`
- Nombres: `week02_beta_bernoulli.ipynb`, `week02_thresholding_metrics.ipynb`
- Celdas limpias (sin outputs gigantes) y semilla fija donde aplique.
- Commits pequeños con mensajes claros.
- Reproducible: `requirements.txt` o `environment.yml` (si ya lo estamos usando).

Cierre

- Bayes = prior $\times$ likelihood $\rightarrow$ posterior (actualización).
- Conjugación = actualizaciones simples (conteos $\rightarrow$ parámetros).
- Predicción Bayesiana integra incertidumbre $\Rightarrow$ menos sobreconfianza.
- En clasificación: el score no es la decisión; la decisión depende del umbral y del objetivo.

Semana 3 — Probabilidad (modelos multivariados)

De describir dependencia a inferir faltantes: $\Sigma/R \to$ MVN $\to$ Mahalanobis $\to$ condicionamiento $\to$ imputación (Iris) $\to$ GMM

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 3

Lecturas sugeridas (para estos temas)

- **Capítulo 3 — Probability: Multivariate Models**
  - **3.1.1** Covarianza
  - **3.1.2** Correlación
  - **3.1.3** No correlación $\not\Rightarrow$ independencia
  - **3.1.4** Correlación $\not\Rightarrow$ causalidad
  - **3.1.5** Paradoja de Simpson
  - **3.2.1** Definición de la MVN
  - **3.2.2** Distancia de Mahalanobis
  - **3.2.3** Marginales y condicionales de una MVN *
  - **3.2.4** Ejemplo: condicionamiento en una Gaussiana 2D
  - **3.2.5** Ejemplo: imputación de valores faltantes *
  - **3.3.1** Regla de Bayes para gaussianas
  - **3.5.1** Modelos de mezcla gaussianos (GMM)

Temario de la Semana 3 (qué cubrimos hoy)

- **Teoría (PML)**
  - **1) Describir dependencia:** $\hat{\Sigma}$ y $\hat{R}$ **(3.1.1–3.1.2)**
  - **2) Interpretar con cuidado:** $\rho = 0 \not\Rightarrow$ independencia; correlación $\not\Rightarrow$ causalidad; Simpson **(3.1.3–3.1.5)**
  - **3) Modelar:** MVN $\mathcal{N}(\mu, \Sigma)$ como baseline **(3.2.1)**
  - **4) Geometría/probabilidad:** Mahalanobis (qué tan "raro" es un punto) **(3.2.2)**
  - **5) Inferir:** marginales/condicionales y condicionamiento por bloques **(3.2.3)**
  - **6) Aplicación:** imputación (Iris) como $\mathbb{E}[X_m \mid X_o]$ **(3.2.5)**
  - **7) Extensión:** GMM para multimodalidad **(3.5.1)**
- **Lab NumPy**
  - **Condicionamiento en una MVN** (bloques + fórmulas)
  - **Imputación:** $\mathbb{E}[x_{\text{miss}} \mid x_{\text{obs}}]$
  - **Evaluación:** MSE en valores ocultados
- **Lab sklearn**
  - **GaussianMixture:** ajustar un GMM en 2D
  - **Leer parámetros:** weights, means, covariances
  - **Visualización:** puntos + elipses por componente
- **Entregable**

Hilo conductor de la semana (Iris)

- Vamos a usar **Iris** como ejemplo mental: 4 variables (sépalo/pétalo).
- **Pregunta 1 (describir):** ¿qué variables se mueven juntas? $\Rightarrow \hat{\Sigma}$, $\hat{R}$.
- **Pregunta 2 (modelar):** si aproximamos con una MVN, ¿cómo se ve la nube? $\Rightarrow$ geometría de Σ.
- **Pregunta 3 (medir rareza):** ¿qué puntos son atípicos bajo ese modelo? $\Rightarrow$ Mahalanobis.
- **Pregunta 4 (inferir faltantes):** si falta una variable, ¿qué valor es más plausible? $\Rightarrow$ condicionamiento $\rightarrow$ imputación.
- **Pregunta 5 (ir más allá):** si hay varias "nubes", ¿cómo lo modelamos? $\Rightarrow$ GMM.

Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

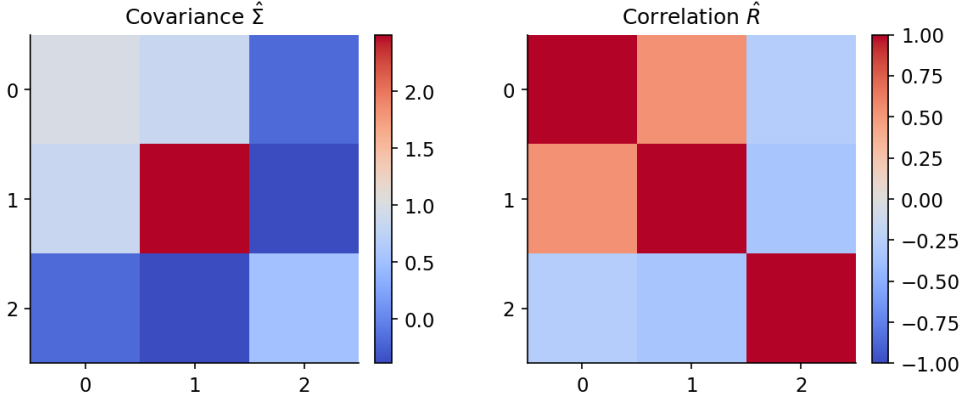
Punto de partida: vector aleatorio, media y covarianza

- En datos reales (p. ej. Iris), cada fila es un vector $x \in \mathbb{R}^D$ (aquí $D = 4$).
- Modelamos X como vector aleatorio: $X \in \mathbb{R}^D$.
- **Media**: $\mu = \mathbb{E}[X]$ (valor "típico" por variable).
- **Covarianza**: $\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^\top]$ (escala y dependencia lineal).
- **Entrada**: $\Sigma_{ij} = \mathrm{Cov}(X_i, X_j)$ indica si dos variables tienden a subir/bajar juntas.

Construcción en práctica: $\hat{\Sigma}$ y $\hat{R}$ (pasos)

1. **Paso 1 — datos:** Tenemos n observaciones $x_1, \ldots, x_n$ (filas). Puede ser Iris (4 variables) o un ejemplo 3D.
2. **Paso 2 — covarianza muestral:** Estimamos $\hat{\Sigma}_{ij}$ como el promedio centrado de productos de desviaciones:
$$\hat{\Sigma}_{ij} = \frac{1}{n-1} \sum_{k=1}^{n} (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j), \quad \bar{x}_i = \frac{1}{n} \sum_{k=1}^{n} x_{ki}.$$
Diagonal = varianza muestral; fuera de diagonal = covarianza (nota: *unidades mezcladas*).
3. **Paso 3 — correlación:** Estandarizamos a escala $[-1, 1]$:
$$\hat{R}_{ij} = \frac{\hat{\Sigma}_{ij}}{\sqrt{\hat{\Sigma}_{ii}\hat{\Sigma}_{jj}}}.$$
4. **Lectura rápida:** usa $\hat{R}$ para comparar dependencias; usa $\hat{\Sigma}$ para cálculos (geometría y condicionamiento).

$\hat{\Sigma}$ vs $\hat{R}$: mismo dato, dos "lentes"

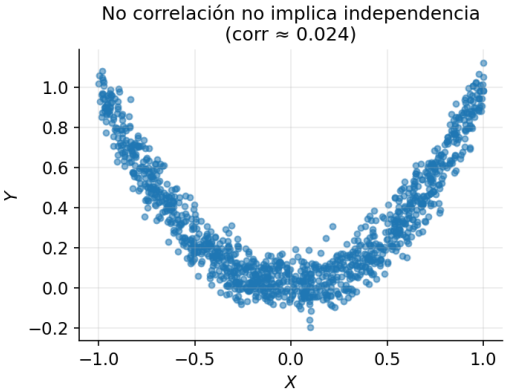


Idea: la izquierda conserva escala (varianzas grandes dominan); la derecha estandariza y hace comparables las relaciones lineales.

Izquierda: $\hat{\Sigma}$. Derecha: $\hat{R}$.

Alerta 1: $\rho = 0$ no implica independencia

- **Mensaje**: correlación mide *dependencia lineal*. Puede haber relación no lineal con $\rho \approx 0$.
- **Hecho**: $\mathrm{Cov}(X, Y) = 0 \not\Rightarrow X \perp Y$ (salvo casos especiales como gaussianas).
- **Traducción práctica**: "no hay correlación" $\neq$ "no hay relación".



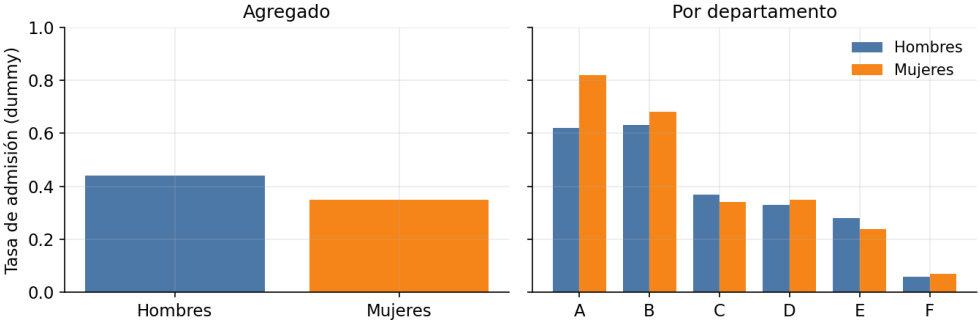
Alerta 2: correlación no implica causalidad

- Una correlación puede venir de **confusores**, **sesgo de selección** o **causalidad invertida**.
- **Pregunta guía**: ¿qué variable podría estar afectando a ambas? ¿qué estoy dejando fuera?

Paradoja de Simpson: cuando el agregado engaña

- **Definición**: una tendencia en datos **agregados** cambia (o se invierte) al **estratificar** por una tercera variable.
- **Ejemplo (Berkeley, 1973)**: al controlar por departamento, la interpretación cambia.
  - **Agregado**: parece haber sesgo.
  - **Estratificado**: aparece el rol del confusor (departamento).

Paradoja de Simpson: visualización y “checklist mental”



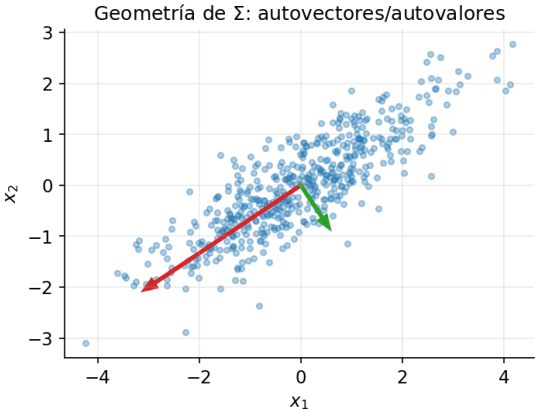
- **Checklist**: antes de concluir, pregunta “¿qué variable podría estar mezclando grupos?”.
- Ejemplos de estratos: departamento, cohorte, nivel educativo, hospital/escuela, etc.

MVN: un baseline probabilístico para vectores

- Idea: aproximar una nube de datos con una sola *elipse* (2D) / *elipsoide* (D).
- Notación: $X \sim \mathcal{N}(\mu, \Sigma)$.
- μ: centro; Σ: escala + orientación (forma).
- Esto nos da dos cosas: **geometría** (elipses) y **cálculo cerrado** (condicionales).

Geometría de Σ: cómo "lee" la MVN la nube de datos

- En una MVN, las curvas de igual densidad son *elipses*/*elipsoides*.
- **Autovectores**: direcciones principales (ejes) de la nube.
- **Autovalores**: dispersión en cada dirección (eje $\propto \sqrt{\lambda}$).
- **Lectura práctica**: Σ codifica *escala* y *correlación* en una sola matriz.

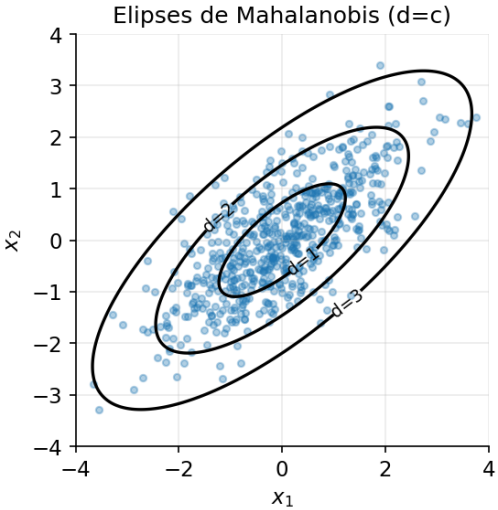


Distancia de Mahalanobis: qué es y para qué sirve

- **Definición:**

$$d_M(x, \mu) = \sqrt{(x-\mu)^\top \Sigma^{-1} (x-\mu)}.$$

- **Intuición:** mide "lejanía" en unidades que respetan *escala* y *correlación*.
- **Sirve para:**
  - detectar *atípicos* bajo una MVN (outliers),
  - entender el *log-likelihood* gaussiano (el término cuadrático),
  - conectar con GMM: cada componente usa su Σ_k para medir cercanía.
- **Puente:** la geometría que define d_M es la misma que usamos al **condicionar** (siguiente bloque).



De “medir rareza” a “inferir faltantes”

- Con μ y Σ ya podemos:
  - evaluar qué tan plausible es un punto (vía d_M),
  - y también **inferir** partes faltantes de un vector.
- La herramienta clave es que en la MVN, **marginales** y **condicionales** siguen siendo gaussianas.

Marginales y condicionales en una MVN (la pieza que habilita todo)

- **Marginal**: si te quedas con algunas coordenadas, sigues en el mundo gaussiano.
- **Condicional**: si conoces parte del vector, la distribución del resto también es gaussiana.
- **Consecuencia**: podemos calcular $\mathbb{E}[X_{\text{faltante}} \mid X_{\text{observado}}]$ en forma cerrada $\Rightarrow$ imputación.

Condicionamiento por bloques: la receta (MVN)

- **Notación**: a = variables *observadas*; b = variables *faltantes* (o las que queremos predecir).
- Partición:

$$x = \begin{pmatrix} x_a \\ x_b \end{pmatrix}, \quad \mu = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}.$$

- Resultado: $X_b \mid X_a = a \sim \mathcal{N}(\mu_{b|a}, \Sigma_{b|a})$.
- Fórmulas:

$$\mu_{b|a} = \mu_b + \Sigma_{ba}\Sigma_{aa}^{-1}(a - \mu_a), \qquad \Sigma_{b|a} = \Sigma_{bb} - \Sigma_{ba}\Sigma_{aa}^{-1}\Sigma_{ab}.$$

- Nota: $\Sigma_{b|a}$ no depende de a. Para imputación "simple", usamos la media $\mu_{b|a}$.

Condicionamiento por bloques: ejemplo numérico (D=5)

Ejemplo: observamos 3 variables (a) y faltan 2 (b). Así "se ven" μ y Σ.

- Vector de medias: $\mu = \begin{pmatrix} 2{,}1 \\ 1{,}8 \\ 3{,}0 \\ 4{,}2 \\ 5{,}5 \end{pmatrix}$ (shape $(5,)$).
- Matriz de covarianza Σ (shape (5×5)):

$$\Sigma = \begin{pmatrix} 1{,}2 & 0{,}5 & 0{,}3 & 0{,}4 & 0{,}2 \\ 0{,}5 & 0{,}9 & 0{,}2 & 0{,}3 & 0{,}1 \\ 0{,}3 & 0{,}2 & 1{,}5 & 0{,}6 & 0{,}4 \\ 0{,}4 & 0{,}3 & 0{,}6 & 2{,}0 & 0{,}8 \\ 0{,}2 & 0{,}1 & 0{,}4 & 0{,}8 & 1{,}8 \end{pmatrix}$$

- Partición: $a =$ índices $[0, 1, 2]$ (observadas), $b =$ índices $[3, 4]$ (faltantes).

Condicionamiento por bloques: ejemplo numérico (D=5) — subbloques

- **Partir** μ en observado/faltante:

$$\mu_a = \begin{pmatrix} 2{,}1 \\ 1{,}8 \\ 3{,}0 \end{pmatrix} \quad \text{(shape } (3,)\text{)}, \qquad \mu_b = \begin{pmatrix} 4{,}2 \\ 5{,}5 \end{pmatrix} \quad \text{(shape } (2,)\text{)}.$$

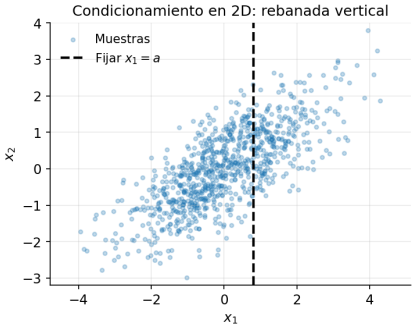
- **Partir** Σ en 4 bloques (observado-observado, observado-faltante, etc.):

$$\Sigma_{aa} = \begin{pmatrix} 1{,}2 & 0{,}5 & 0{,}3 \\ 0{,}5 & 0{,}9 & 0{,}2 \\ 0{,}3 & 0{,}2 & 1{,}5 \end{pmatrix} \quad \text{(shape } (3 \times 3)\text{)},$$

$$\Sigma_{ab} = \begin{pmatrix} 0{,}4 & 0{,}2 \\ 0{,}3 & 0{,}1 \\ 0{,}6 & 0{,}4 \end{pmatrix} \quad \text{(shape } (3 \times 2)\text{)}, \qquad \Sigma_{ba} = \Sigma_{ab}^{\top}, \quad \Sigma_{bb} = \begin{pmatrix} 2{,}0 & 0{,}8 \\ 0{,}8 & 1{,}8 \end{pmatrix} \quad \text{(shape } (2 \times 2)\text{)}.$$

- **Lectura**: Σ_{aa} es lo que invertimos; Σ_{ba} "transporta" información de observado $\rightarrow$ faltante.
- Para imputar: $\mu_{b|a} = \mu_b + \Sigma_{ba}\Sigma_{aa}^{-1}(x_a - \mu_a)$ (shape $(2,)$).

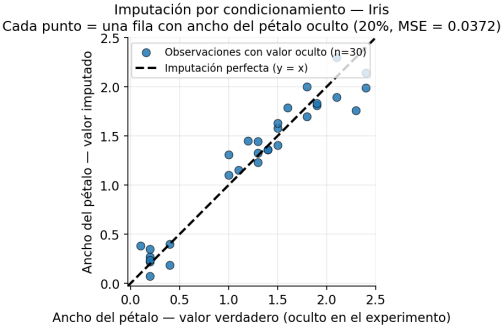
Condicionamiento en 2D: intuición visual (rebanar la elipse)



- En 2D: observado $a = x_1$, faltante $b = x_2$.
- Fijar $x_1 = a$ "rebana" la nube: queda una normal 1D para $x_2 \mid x_1 = a$.

Imputación (Iris) = condicionamiento + evaluación honesta

- **Idea**: tratar "faltante" como X_m y lo observado como X_o.
- **Regla**: imputación determinista $x_m \leftarrow \mathbb{E}[X_m \mid X_o = x_o]$ (media condicional).
- **Experimento**: ocultamos ancho del pétalo en 20 % de filas; estimamos (μ, Σ) con las filas observadas; imputamos y comparamos.
- **Gráfica**: cada punto azul = una celda ocultada (verdadero vs imputado). Diagonal $y = x$ = ideal.
- **Métrica**: MSE **solo** donde ocultaste; compara contra baseline (media marginal por columna).



¿Por qué no basta una sola MVN? (puente a mixtures)

- Una MVN = una sola "nube elíptica".
- En Iris, distintas especies sugieren **más de una nube** (multimodalidad).
- Solución: **mezcla** = suma ponderada de varias MVNs.

GMM: varias elipses para explicar una nube compleja

- Modelo:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \sum_k \pi_k = 1,\ \pi_k \geq 0.$$

- En 2D, cada componente induce una elipse; el modelo suma "nubes".
- Interpretación: π_k (peso), μ_k (centro), Σ_k (forma).
- Conexión: "cercanía" a cada componente se relaciona con Mahalanobis y su log-likelihood.

Cierre: qué te llevas de Week03

- $\hat{\Sigma}$ conserva escala; $\hat{R}$ estandariza para comparar dependencias.
- Interpretación cuidadosa: $\rho = 0 \not\Rightarrow$ independencia; correlación $\not\Rightarrow$ causalidad; cuidado con Simpson.
- MVN + Σ da geometría; Mahalanobis mide "rareza" bajo esa geometría.
- Condicionamiento MVN es cerrado $\Rightarrow$ imputación como $\mathbb{E}[X_m \mid X_o]$.
- Si hay varias nubes, GMM (mezcla de elipses) es una extensión natural.

Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

Lab NumPy — objetivo general

- Implementar la **media condicional** de una MVN por bloques (la receta de Teoría).
- Aplicarla a **imputación** (p. ej. Iris: ocultar una columna y recuperarla).
- Evaluar sin engañarse: MSE **solo** en valores ocultados + baseline.
- Reproducibilidad: semilla fija, reporta N y % faltantes.

Lab NumPy — notación y shapes (traducción Teoría → código)

- `mu (D,)`: vector de medias; `mu[i]` = media de la variable i.
- `Sigma (D,D)`: covarianza; `Sigma[i,j]` = $\mathrm{cov}(X_i, X_j)$.
- `idx_a / idx_b`: índices de observado/faltante (en Iris, $D = 4$ y puedes ocultar "petal width").
- `x_a`: valores observados de una fila; es el "a" numérico de esa observación.
- Fórmula por fila (media condicional): $\mu_{b|a} = \mu_b + \Sigma_{ba}\Sigma_{aa}^{-1}(x_a - \mu_a)$.

Lab NumPy — Pseudocódigo 1: partir en bloques

```
# INPUT:
#   mu:    shape (D,)    # vector de medias
#   Sigma: shape (D,D)   # matriz de covarianza
#   idx_a: índices observados
#   idx_b: índices faltantes

mu_a = mu[idx_a]                          # (Da,)
mu_b = mu[idx_b]                          # (Db,)
S_aa = Sigma[np.ix_(idx_a, idx_a)]        # (Da,Da)
S_ab = Sigma[np.ix_(idx_a, idx_b)]        # (Da,Db)
S_ba = Sigma[np.ix_(idx_b, idx_a)]        # (Db,Da)
S_bb = Sigma[np.ix_(idx_b, idx_b)]        # (Db,Db)

# CHEQUEO: S_aa debe ser invertible (si no, regulariza: S_aa += eps*I)
```

Lab NumPy — Pseudocódigo 2: media condicional → imputación

```
# Precompute una vez (fuera del loop por filas):
S_aa_inv = inv(S_aa)   # costoso -> hazlo una sola vez

# Para cada fila i con faltantes en idx_b:
x_a = X[i, idx_a]      # valores observados de la fila (Da,)

# Fórmula MVN:
# mu_{b|a} = mu_b + S_ba @ inv(S_aa) @ (x_a - mu_a)
mu_b_given_a = mu_b + S_ba @ S_aa_inv @ (x_a - mu_a)  # (Db,)

# Imputación determinista (media condicional):
X_imputed[i, idx_b] = mu_b_given_a

# Opcional (imputación estocástica):
# S_{b|a} = S_bb - S_ba @ inv(S_aa) @ S_ab
```

Lab NumPy — Pseudocódigo 3: evaluación correcta (MSE)

```
# 1) Guarda el dato completo
X_true = X.copy()                    # (N,D)

# 2) Genera máscara de celdas a ocultar (True = oculto)
mask = make_mask(...)                # (N,D) boolean

# 3) Oculta y luego imputa
X_missing = X_true.copy()
X_missing[mask] = NaN
X_imputed = impute(X_missing)        # tu función del pseudocódigo 2

# 4) Evalúa solo donde ocultaste (para no "hacer trampa")
mse = mean( (X_true[mask] - X_imputed[mask])**2 )

# Baseline: imputar por media marginal (media de columna)
```

Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

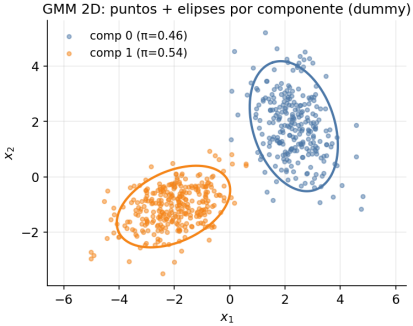
Lab sklearn — objetivo general

- Ajustar un **GMM** en 2D (elige K, fija semilla).
- Leer parámetros: pesos, medias y covarianzas $\Rightarrow$ "varias elipses".
- Visualizar: puntos + elipses por componente.
- Interpretar: 2–3 frases (¿qué parte de la nube captura cada componente?).

GMM: cómo interpretar π_k, μ_k, Σ_k

- π_k: proporción de datos que "explica" el componente k.
- μ_k: centro del componente.
- Σ_k: forma/orientación (elipse) del componente.
- **Pregunta guía:** ¿el componente captura estructura real o está "partiendo" artificialmente una sola nube?

GMM en 2D: visualización (puntos + elipses)



- Elipses = covarianza por componente; colores = componente más probable.
- Lectura: cada elipse es una "nube" gaussiana; el mixto suma varias nubes.

Lab sklearn — Checklist (mínimo indispensable)

```
# INPUT:
#   X: shape (N,2)   # datos 2D

# PASOS:
# 1) ajustar GaussianMixture(K)
# 2) leer weights_, means_, covariances_
# 3) predecir componente / probabilidades
# 4) graficar puntos + elipses

# OUTPUT:
#   figura 2D + breve interpretación
```

Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

Entregable — qué se evalúa

- **Notebook 1 (NumPy):** condicionamiento por bloques $\rightarrow$ imputación $\rightarrow$ MSE (con baseline).
- **Notebook 2 (sklearn):** GMM 2D $\rightarrow$ parámetros $\rightarrow$ visualización + interpretación breve.
- **Reproducible:** semilla, N, % faltantes (si aplica), y una conclusión corta por notebook.